

Exploration in RL: Multi-armed Bandit

Kianté Brantley
CS 1840 Fall 2026

Announcements

HW1 is out!

Check the course website for Office Hours:

<https://harvard-cs-1840.github.io/2026/>

Recap: Value Iteration vs Policy Iteration

Policy Iteration:

1. Initialization: $\pi^0 : S \mapsto \mathcal{A}(A)$

2. For $t = 0 \dots$,

→ compute $Q^{\pi^t}(s, a), \forall s, a$

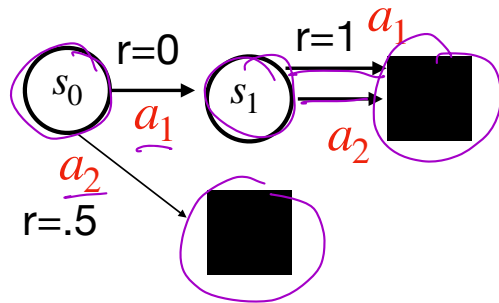
$\pi^{t+1}(s) := \arg \max_a Q^{\pi^t}(s, a), \forall s$

Recap: Value Iteration vs Policy Iteration

Policy Iteration:

1. Initialization: $\pi^0 : S \mapsto \Delta(A)$
2. For $t = 0 \dots$,
compute $Q^{\pi^t}(s, a), \forall s, a$
 $\pi^{t+1}(s) := \arg \max_a Q^{\pi^t}(s, a), \forall s$

Action Gap:



Recap: Value Iteration vs Policy Iteration

Policy Iteration:

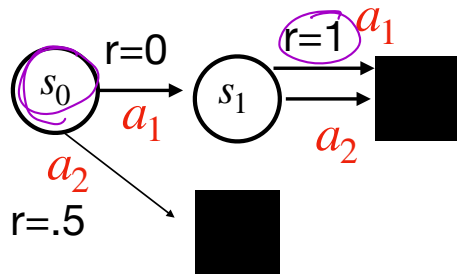
1. Initialization: $\pi^0 : S \mapsto \Delta(A)$
2. For $t = 0 \dots$,
compute $Q^{\pi^t}(s, a), \forall s, a$
 $\pi^{t+1}(s) := \arg \max_a Q^{\pi^t}(s, a), \forall s$

$$\pi_0(s_0) = \underline{a_2}$$

$$Q^{\pi_0}(s_1, \cdot) = \textcircled{1}$$

$$\left[\begin{array}{l} Q^{\pi_0}(s_0, \textcircled{a_1}) = .99 \\ Q^{\pi_0}(s_0, a_2) = 0.5 \end{array} \right]$$

Action Gap:



Recap: Value Iteration vs Policy Iteration

Policy Iteration:

1. Initialization: $\pi^0 : S \mapsto \Delta(A)$

2. For $t = 0 \dots$,

compute $Q^{\pi^t}(s, a), \forall s, a$

$\pi^{t+1}(s) := \arg \max_a Q^{\pi^t}(s, a), \forall s$

$\pi^0 \rightarrow \pi^1$

$$\pi_0(s_0) = a_2$$

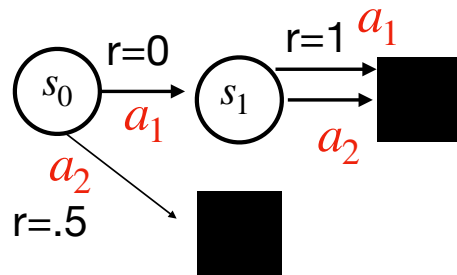
$$Q^{\pi_0}(s_1, \cdot) = 1$$

$$Q^{\pi_0}(s_0, a_1) = .99$$

$$Q^{\pi_0}(s_0, a_2) = 0.5$$

$$\pi_1(s_0) = a_1$$

Action Gap:



Recap: Value Iteration vs Policy Iteration

Value Iteration:

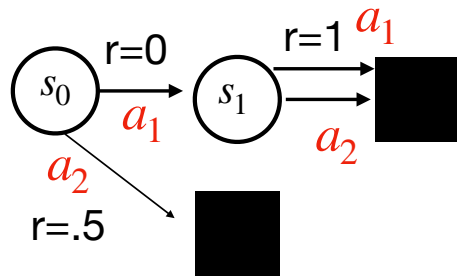
1. Initialization: Q^0

2. For $t = 0 \dots$,

Iterate until convergence:

$$\underline{Q^{t+1}}(s, a) = \underline{r(s, a)} + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \max_{a'} Q^t(s', a'), \forall s, a$$

Action Gap:



Recap: Value Iteration vs Policy Iteration

Value Iteration:

1. Initialization: Q^0

2. For $t = 0 \dots$,

Iterate until convergence:

$$Q^{t+1}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \max_{a'} Q^t(s', a'), \forall s, a$$

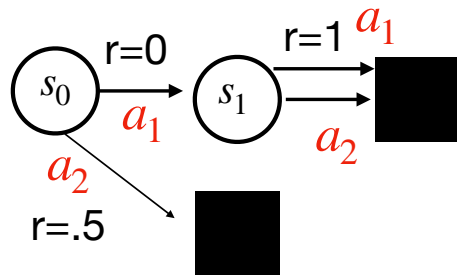
$$Q^0 = 0$$

$$Q^1(s_0, a_1) = 0 + \gamma Q^0 = 0$$

$$Q^1(s_0, a_2) = 0.5$$

$$Q^1(s_1, \cdot) = 1$$

Action Gap:



Recap: Value Iteration vs Policy Iteration

Value Iteration:

1. Initialization: Q^0

2. For $t = 0 \dots$,

Iterate until convergence:

$$Q^{t+1}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \max_{a'} Q^t(s', a'), \forall s, a$$

$$Q^0 = 0$$

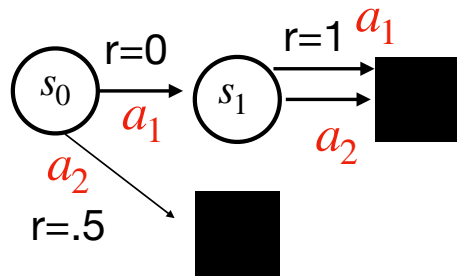
$$Q^1(s_0, a_0) = 0 + \gamma Q^0 = 0$$

$$Q^1(s_0, a_2) = 0.5$$

$$Q^1(s_1, \cdot) = 1$$

$$\pi_1(s_0) = a_2$$

Action Gap:



Recap: Finite horizon Markov Decision Process

$$\mathcal{M} = \{S, A, r, P, \underbrace{H}_{\text{Horizon}}, \underbrace{\mu_0}_{\Delta}\},$$
$$\underbrace{r}_{\Delta} : S \times A \mapsto [0,1], \underbrace{H}_{\Delta} \in \mathbb{N}^+, P : S \times A \mapsto \Delta(S), s_0 \sim \mu_0$$

Note that in finite horizon setting, we will consider **time-dependent policies**, i.e.,

$$\pi := \{\underbrace{\pi_0}_{\Delta}, \underbrace{\pi_1}_{\Delta}, \dots, \pi_{H-1}\}, \pi_h : S \mapsto A, \forall h$$

Recap: Finite horizon Markov Decision Process

$$\mathcal{M} = \{S, A, r, P, H, \mu_0\},$$

$$r : S \times A \mapsto [0, 1], H \in \mathbb{N}^+, P : S \times A \mapsto \Delta(S), s_0 \sim \mu_0$$

Note that in finite horizon setting, we will consider **time-dependent policies**, i.e.,

$$\pi := \{\pi_0, \pi_1, \dots, \pi_{H-1}\}, \pi_h : S \mapsto A, \forall h$$

Policy interacts with the MDP as follows:

$$\tau = \{s_0, a_0, s_1, a_1, \dots, s_{H-1}, a_{H-1}\}, s_0 \sim \mu_0, a_0 = \pi_0(s_0), s_1 \sim P(\cdot | s_0, a_0), a_1 = \pi_1(s_1), \dots$$

Finishing Up From Last Lecture



1: An Iterative Algorithm: Policy Iteration



2: Convergence? How fast?

3: A new model: Finite horizon MDP

Compute Optimal Policy via DP

$$\pi^\star = \{\pi_0^\star, \pi_1^\star, \dots, \pi_{H-1}^\star\}$$



Compute Optimal Policy via DP

$$\pi^\star = \{\pi_0^\star, \pi_1^\star, \dots, \pi_{H-1}^\star\}$$

We use Dynamic Programming, and do DP backward in time; start at $H - 1$

Compute Optimal Policy via DP

$$\pi^{\star} = \{\pi_0^{\star}, \pi_1^{\star}, \dots, \pi_{H-1}^{\star}\}$$

We use Dynamic Programming, and do DP backward in time; start at $H - 1$

$$Q_{H-1}^{\star}(s, a) = r(s, a)$$



Compute Optimal Policy via DP

$$\pi^{\star} = \{\pi_0^{\star}, \pi_1^{\star}, \dots, \pi_{H-1}^{\star}\}$$

We use Dynamic Programming, and do DP backward in time; start at $H - 1$

$$Q_{H-1}^{\star}(s, a) = r(s, a) \quad \pi_{H-1}^{\star}(s) = \arg \max_a Q_{H-1}^{\star}(s, a)$$

Compute Optimal Policy via DP

$$\pi^{\star} = \{\pi_0^{\star}, \pi_1^{\star}, \dots, \pi_{H-1}^{\star}\}$$

We use Dynamic Programming, and do DP backward in time; start at $H - 1$

$$Q_{H-1}^{\star}(s, a) = r(s, a) \quad \pi_{H-1}^{\star}(s) = \arg \max_a Q_{H-1}^{\star}(s, a) \quad V_{H-1}^{\star}(s) = \max_a Q_{H-1}^{\star}(s, a)$$

Compute Optimal Policy via DP

$$\pi^{\star} = \{\pi_0^{\star}, \pi_1^{\star}, \dots, \pi_{H-1}^{\star}\}$$

We use Dynamic Programming, and do DP backward in time; start at $H - 1$

$$Q_{H-1}^{\star}(s, a) = r(s, a) \quad \pi_{H-1}^{\star}(s) = \arg \max_a Q_{H-1}^{\star}(s, a) \quad V_{H-1}^{\star}(s) = \max_a Q_{H-1}^{\star}(s, a)$$

Now assume that we have already computed $V_{h+1}^{\star}$, $h \leq H - 2$
(i.e., we know how to perform optimally starting at $h + 1$)

Compute Optimal Policy via DP

$$\pi^\star = \{\pi_0^\star, \pi_1^\star, \dots, \pi_{H-1}^\star\}$$

We use Dynamic Programming, and do DP backward in time; start at $H - 1$

$$Q_{H-1}^\star(s, a) = r(s, a) \quad \pi_{H-1}^\star(s) = \arg \max_a Q_{H-1}^\star(s, a) \quad V_{H-1}^\star(s) = \max_a Q_{H-1}^\star(s, a)$$

Now assume that we have already computed $V_{h+1}^\star$, $h \leq H - 2$

(i.e., we know how to perform optimally starting at $h + 1$)

$$Q_h^\star(s, a) = r(s, a) + \mathbb{E}_{s' \sim P(\cdot | s, a)} V_{h+1}^\star(s')$$

Compute Optimal Policy via DP

$$\pi^{\star} = \{\pi_0^{\star}, \pi_1^{\star}, \dots, \pi_{H-1}^{\star}\}$$

We use Dynamic Programming, and do DP backward in time; start at $H - 1$

$$Q_{H-1}^{\star}(s, a) = r(s, a) \quad \pi_{H-1}^{\star}(s) = \arg \max_a Q_{H-1}^{\star}(s, a) \quad V_{H-1}^{\star}(s) = \max_a Q_{H-1}^{\star}(s, a)$$

Now assume that we have already computed $V_{h+1}^{\star}$, $h \leq H - 2$

(i.e., we know how to perform optimally starting at $h + 1$)

$$Q_h^{\star}(s, a) = r(s, a) + \mathbb{E}_{s' \sim P(\cdot | s, a)} V_{h+1}^{\star}(s') \quad \pi_h^{\star}(s) = \arg \max_a Q_h^{\star}(s, a)$$

The need for Exploration in RL:

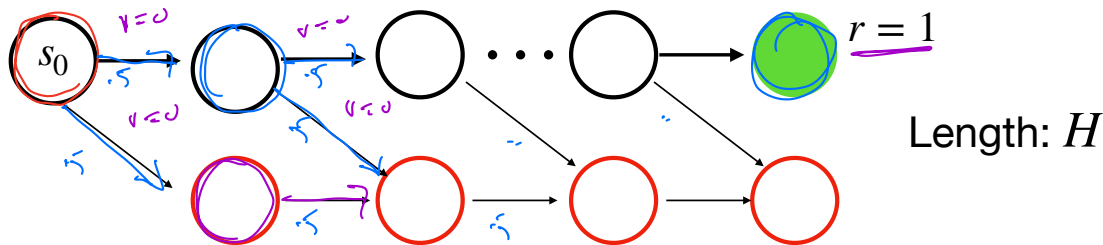
The need for Exploration in RL:

What is the probability of a random policy generating a trajectory that hits the goal?

The need for Exploration in RL:

The Combination Lock Example (i.e., the sparse reward problem)

- (1) We have reward zero everywhere except at the goal (the right end);
- (2) Every black node, one of the two actions will lead the agent to the dead state (red)



What is the probability of a random policy generating a trajectory that hits the goal?

$$\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdots = \left(\frac{1}{2}\right)^H$$

Exploration!

We need to perform systematic exploration,
i.e., remember where we visited, and purposely
try to visit unexplored regions..

What we will do here:

Study Exploration in a very simple MDP:

$$\mathcal{M} = \{s_0, \{a_1, \dots, a_K\}, H = 1, R\}$$


i.e., MDP with one state, one-step transition, and K actions

This is also called Multi-armed Bandits

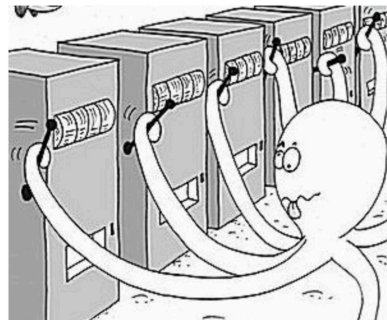
Outline

1. Introduction of MAB
2. Attempt 1: Greedy Algorithm (a bad algorithm)
3. Attempt 2: Explore-Then-Commit

Intro to MAB

Setting:

We have K many arms: $1, \dots, K$

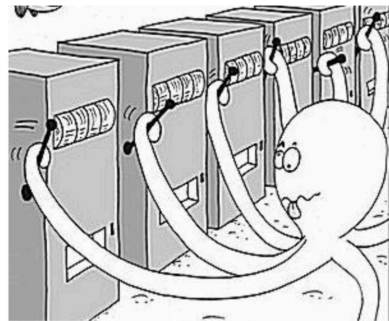


Intro to MAB

Setting:

We have K many arms: $1, \dots, K$

Each arm has an unknown reward distribution, i.e., $\nu_k \in \Delta([0,1])$,
w/ mean $\mu_k = \mathbb{E}_{r \sim \nu_k}[r]$



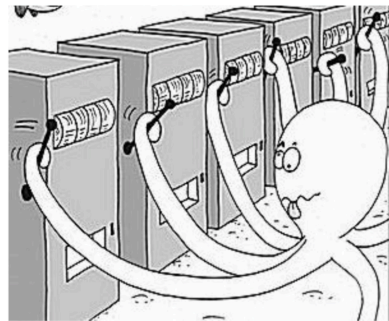
Intro to MAB

Setting:

We have K many arms: $1, \dots, K$

Each arm has an unknown reward distribution, i.e., $\nu_k \in \Delta([0,1])$,
w/ mean $\mu_k = \mathbb{E}_{r \sim \nu_k}[r]$

Example: arm k has a Bernoulli distribution ν_k w/ mean $\mu_k := p$:



Intro to MAB

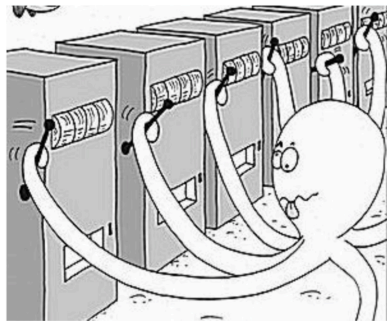
Setting:

We have K many arms: $1, \dots, K$

Each arm has an unknown reward distribution, i.e., $\nu_k \in \Delta([0,1])$,
w/ mean $\mu_k = \mathbb{E}_{r \sim \nu_k}[r]$

Example: arm k has a Bernoulli distribution ν_k w/ mean $\mu_k := p$:

Every time we pull arm k , we observe an i.i.d reward $r = \begin{cases} 1 & \text{w/ prob } \mu_k \\ 0 & \text{w/ prob } 1 - \mu_k \end{cases}$



Intro to MAB

Applications on online advertisement:



Arms correspond to Ads

Each arm has **click-through-rate**
(CTR): probability of getting clicked
(unknown)

Intro to MAB

Applications on online advertisement:



A learning system aims to maximize CTR in a long run:

Arms correspond to Ads

Each arm has **click-through-rate** (CTR): probability of getting clicked (unknown)

Intro to MAB

Applications on online advertisement:



Arms correspond to Ads

Each arm has **click-through-rate** (CTR): probability of getting clicked (unknown)

A learning system aims to maximize CTR in a long run:

1. **Try** an Ad (pull an arm)

Intro to MAB

Applications on online advertisement:



Arms correspond to Ads

Each arm has **click-through-rate** (CTR): probability of getting clicked (unknown)

A learning system aims to maximize CTR in a long run:

1. **Try** an Ad (pull an arm)
2. **Observe** if it is clicked (see a zero-one **reward**)

Intro to MAB

Applications on online advertisement:



Arms correspond to Ads

Each arm has **click-through-rate** (CTR): probability of getting clicked (unknown)

A learning system aims to maximize CTR in a long run:

1. **Try** an Ad (pull an arm)
2. **Observe** if it is clicked (see a zero-one **reward**)
3. **Update**: Decide what ad to recommend for next round

Intro to MAB

More formally, we have the following interactive learning process:

For $t = 0 \rightarrow T - 1$



Intro to MAB

More formally, we have the following interactive learning process:

For $t = 0 \rightarrow T - 1$

1. Learner pulls arm $a_t \in \{1, \dots, K\}$


Intro to MAB

More formally, we have the following interactive learning process:

For $t = 0 \rightarrow T - 1$

(# based on historical information)

1. Learner pulls arm $a_t \in \{1, \dots, K\}$

Intro to MAB

More formally, we have the following interactive learning process:

For $t = 0 \rightarrow T - 1$

(# based on historical information)

1. Learner pulls arm $a_t \in \{1, \dots, K\}$

2. Learner observes an i.i.d reward $r_t \sim \nu_{a_t}$ of arm a_t



Intro to MAB

More formally, we have the following interactive learning process:

For $t = 0 \rightarrow T - 1$

(# based on historical information)

1. Learner pulls arm $a_t \in \{1, \dots, K\}$
2. Learner observes an i.i.d reward $r_t \sim \nu_{a_t}$ of arm a_t

Note: each iteration, we do not observe rewards of arms that we did not try

Intro to MAB

More formally, we have the following interactive learning process:

For $t = 0 \rightarrow T - 1$

(# based on historical information)

1. Learner pulls arm $a_t \in \{1, \dots, K\}$
2. Learner observes an i.i.d reward $r_t \sim \nu_{a_t}$ of arm a_t

Note: each iteration, we do not observe rewards of arms that we did not try

Note: there is no state s

Intro to MAB

More formally, we have the following learning objective:

$$\boxed{\text{Regret}_T} = \underline{T\mu^\star} - \sum_{t=0}^{T-1} \mu_{a_t} \quad \mu^\star = \max_{k \in [K]} \mu_k$$

Intro to MAB

More formally, we have the following learning objective:

$$\text{Regret}_T = T\mu^\star - \sum_{t=0}^{T-1} \mu_{a_t}$$
$$\mu^\star = \max_{k \in [K]} \mu_k$$

Total expected reward if we
pulled best arm over T rounds

Intro to MAB

More formally, we have the following learning objective:

$$\text{Regret}_T = T\mu^\star - \sum_{t=0}^{T-1} \mu_{a_t}$$

$\mu^\star = \max_{k \in [K]} \mu_k$

Total expected reward if we pulled best arm over T rounds

Total expected reward of the arms we pulled over T rounds

Intro to MAB

More formally, we have the following learning objective:

$$\text{Regret}_T = T\mu^\star - \sum_{t=0}^{T-1} \mu_{a_t}$$

$\mu^\star = \max_{k \in [K]} \mu_k$

Total expected reward if we pulled best arm over T rounds

Total expected reward of the arms we pulled over T rounds

Goal: no-regret, i.e., $\text{Regret}_T/T \rightarrow 0$, as $T \rightarrow \infty$

Intro to MAB

More formally, we have the following learning objective:

$$\text{Regret}_T = T\mu^\star - \sum_{t=0}^{T-1} \mu_{a_t}$$

$\mu^\star = \max_{k \in [K]} \mu_k$

Why not sum the r_t ?

Total expected reward if we pulled best arm over T rounds

Total expected reward of the arms we pulled over T rounds

Goal: no-regret, i.e., $\text{Regret}_T/T \rightarrow 0$, as $T \rightarrow \infty$

Intro to MAB

Why the problem is hard?

Exploration and Exploitation Tradeoff:

Intro to MAB

Why the problem is hard?

Exploration and Exploitation Tradeoff:

Every round, we need to ask ourselves:

Should we pull arms that are less frequently tried in the past (i.e., **explore**),
Or should we commit to the current best arm (i.e., **exploit**)?

Outline



1. Introduction of MAB

2. Attempt 1: Greedy Algorithm (a bad algorithm)

3. Attempt 2: Explore-Then-Commit

Attempt 1: Greedy Algorithm

Alg: try each arm once, and then commit to the one that has the **highest observed** reward

Attempt 1: Greedy Algorithm

Alg: try each arm once, and then commit to the one that has the **highest observed** reward

Q: what could be wrong?

Attempt 1: Greedy Algorithm

Alg: try each arm once, and then commit to the one that has the **highest observed** reward

Q: what could be wrong?

A bad arm (i.e., low μ_i) may generate a high reward by chance!
(recall we have $r \sim \nu$, i.i.d)

Attempt 1: Greedy Algorithm

More concretely, let's say we have two arms a_1, a_2 :

Reward dist for a_1 : w/ prob 60%, $r = 1$; else $r = 0$ $\mu_1 = 0.6$

Reward dist for a_2 : w/ prob 40%, $r = 1$; else $r = 0$ $\mu_2 = 0.4$

Attempt 1: Greedy Algorithm

More concretely, let's say we have two arms a_1, a_2 :

Reward dist for a_1 : w/ prob 60%, $r = 1$; else $r = 0$

Reward dist for a_2 : w/ prob 40%, $r = 1$; else $r = 0$

Clearly a_1 is a better arm!

Attempt 1: Greedy Algorithm

More concretely, let's say we have two arms a_1, a_2 :

Reward dist for a_1 : w/ prob 60%, $r = 1$; else $r = 0$

Reward dist for a_2 : w/ prob 40%, $r = 1$; else $r = 0$

Clearly a_1 is a better arm!

$$(0.4 \times 0.4) = .16$$

But try a_1, a_2 once, with probability 16%, we will observe reward pair $(0,1)$

↑ ↑

$$a_1 \Rightarrow 0.4$$



$$a_2 \Rightarrow 0.4$$

Attempt 1: Greedy Algorithm

More concretely, let's say we have two arms a_1, a_2 :

Reward dist for a_1 : w/ prob 60%, $r = 1$; else $r = 0$

Reward dist for a_2 : w/ prob 40%, $r = 1$; else $r = 0$

Clearly a_1 is a better arm!

$$T \cdot \mu^k = \sum_{t=0}^T \mu^k a_t$$

After round 2

$$.6 - .4 = .2$$
$$\frac{.2(T-2)}{T} \rightarrow .2$$

But try a_1, a_2 once, with probability 16%, we will observe reward pair (0,1)

The greedy alg will pick a_2 —**loosing expected reward 0.2 every time in the future**

Outline:



1. Introduction of MAB



2. Attempt 1: Greedy Algorithm
(a bad algorithm: constant regret)

3. Attempt 2: Explore-Then-Commit

- Algorithm
- Analysis

What lessons we learned from the Greedy Alg:

Due to randomness in the reward distribution, trying each arm once is not enough,
i.e., observed single reward may be far away from the mean

What lessons we learned from the Greedy Alg:

Due to randomness in the reward distribution, trying each arm once is not enough, i.e., observed single reward may be far away from the mean

Q: what's the fix here?

What lessons we learned from the Greedy Alg:

Due to randomness in the reward distribution, trying each arm once is not enough, i.e., observed single reward may be far away from the mean

Q: what's the fix here?

Yes, let's (1) try each arm multiple times, (2) compute the empirical mean of each arm, (3) commit to the one that has the highest empirical mean

What lessons we learned from the Greedy Alg:

Due to randomness in the reward distribution, trying each arm once is not enough, i.e., observed single reward may be far away from the mean

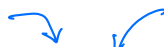
Q: what's the fix here?

Yes, let's (1) try each arm multiple times, (2) compute the empirical mean of each arm, (3) commit to the one that has the highest empirical mean

How many times do we need to try each?

Alg: Explore-Then-Commit (ETC)

rounds # arms



Algorithm hyper parameter $N < T/K$ (we assume $T \gg K$)

↑ # of times we try each arm

For $k = 1 \rightarrow K$: (# Exploration phase)

↑

Alg: Explore-Then-Commit (ETC)

Algorithm hyper parameter $N < T/K$ (we assume $T \gg K$)

For $k = 1 \rightarrow K$: (# Exploration phase)

Pull arm- k N times, observe $\{r_i^{(k)}\}_{i=1}^N \sim \nu_k$

Alg: Explore-Then-Commit (ETC)

Algorithm hyper parameter $N < T/K$ (we assume $T \gg K$)

For $k = 1 \rightarrow K$: (# Exploration phase)

Pull arm- k N times, observe $\{r_i^{(k)}\}_{i=1}^N \sim \nu_k$

Calculate arm k 's empirical mean: $\hat{\mu}_k = \frac{1}{N} \sum_{i=1}^N r_i^{(k)}$

$N \rightarrow \infty$
 $\mu_k \rightarrow \mu^*$

Alg: Explore-Then-Commit (ETC)

Algorithm hyper parameter $N < T/K$ (we assume $T \gg K$)

For $k = 1 \rightarrow K$: (# Exploration phase)

Pull arm- k N times, observe $\{r_i^{(k)}\}_{i=1}^N \sim \nu_k$

Calculate arm k 's empirical mean: $\hat{\mu}_k = \frac{1}{N} \sum_{i=1}^N r_i^{(k)}$

For $t = \underbrace{NK}_{\substack{\text{\# times} \\ \text{we pulled} \\ \text{first arm}}} \rightarrow T - 1$: (# Exploitation phase)

Alg: Explore-Then-Commit (ETC)

Algorithm hyper parameter $N < T/K$ (we assume $T \gg K$)

For $k = 1 \rightarrow K$: (# Exploration phase)

Pull arm- k N times, observe $\{r_i^{(k)}\}_{i=1}^N \sim \nu_k$

Calculate arm k 's empirical mean: $\hat{\mu}_k = \frac{1}{N} \sum_{i=1}^N r_i^{(k)}$

For $t = NK \rightarrow T - 1$: (# Exploitation phase)

Pull the best empirical arm, i.e., $a_t = \arg \max_{k \in [K]} \hat{\mu}_k$

Alg: Explore-Then-Commit (ETC)

Algorithm hyper parameter $N < T/K$ (we assume $T \gg K$)

For $k = 1 \rightarrow K$: (# Exploration phase)

Pull arm- k N times, observe $\{r_i^{(k)}\}_{i=1}^N \sim \nu_k$

Calculate arm k 's empirical mean: $\hat{\mu}_k = \frac{1}{N} \sum_{i=1}^N r_i^{(k)}$

For $t = NK \rightarrow T - 1$: (# Exploitation phase)

Pull the best empirical arm, i.e., $a_t = \arg \max_{k \in [K]} \hat{\mu}_k$

Q: how to set N ?

Outline:



1. Introduction of MAB



2. Attempt 1: Greedy Algorithm
(a bad algorithm: constant regret)



3. Attempt 2: Explore-Then-Commit

- Algorithm
- Analysis

Statistical Tools:

1. Hoeffding inequality (optional, no need to remember or understand it)

$$Y \in \Delta([0,1]) \quad r_1, r_2, \dots, r_N \text{ iid}$$

$$\hat{\mu} = \frac{r_1 + r_2 + \dots + r_N}{N}$$

$$|\hat{\mu} - \mu| \quad \text{where } \mu = \mathbb{E}[r]$$

Statistical Tools:

1. Hoeffding inequality (optional, no need to remember or understand it)

Given a distribution $\nu \in \Delta([0,1])$, and N i.i.d samples

$\{r_i\}_{i=1}^N \sim \mu$, w/ probability at least $1 - \delta$, we have:

$$\left| \sum_{i=1}^N r_i / N - \mu \right| \leq O \left(\sqrt{\frac{\ln(1/\delta)}{N}} \right)$$

$\hat{\mu}$

$\mathbb{E}[r]$
avg

Ignore
constant

$\delta = .001$

w/ prob $1 - \delta = .999$

$$|\hat{\mu} - \mu| \leq O \sqrt{\frac{b}{N}}$$

Statistical Tools:

1. Hoeffding inequality (optional, no need to remember or understand it)

Given a distribution $\nu \in \Delta([0,1])$, and N i.i.d samples

$\{r_i\}_{i=1}^N \sim \mu$, w/ probability at least $1 - \delta$, we have:

$$\left| \sum_{i=1}^N r_i / N - \mu \right| \leq O \left(\sqrt{\frac{\ln(1/\delta)}{N}} \right)$$

i.e., this gives us a confidence interval:

Statistical Tools:

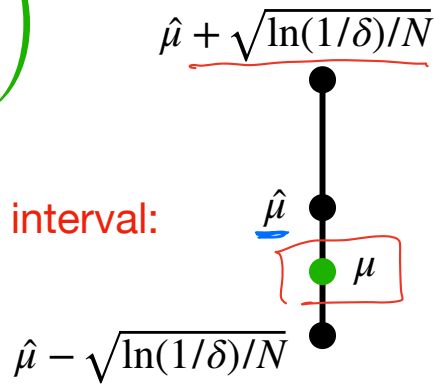
1. Hoeffding inequality (optional, no need to remember or understand it)

Given a distribution $\nu \in \Delta([0,1])$, and N i.i.d samples

$\{r_i\}_{i=1}^N \sim \mu$, w/ probability at least $1 - \delta$, we have:

$$\left| \underbrace{\sum_{i=1}^N r_i / N}_{\hat{\mu}} - \mu \right| \leq O\left(\sqrt{\frac{\ln(1/\delta)}{N}}\right)$$

i.e., this gives us a confidence interval:



Statistical Tools:

2. Union Bound (optional):

$$\mathbb{P}(A \text{ or } B) \leq \mathbb{P}(A) + \mathbb{P}(B)$$

$$\begin{aligned} & \text{area}(\overbrace{A \cap B}^{\text{area}(A \cap B)}) \\ & \leq \text{area}(\underbrace{A}_{\text{area}(A)} + \underbrace{B}_{\text{area}(B)}) \end{aligned}$$

Statistical Tools:

2. Union Bound (optional):

$$\mathbb{P}(A \text{ or } B) \leq \mathbb{P}(A) + \mathbb{P}(B)$$

Combine Hoeffding and Union Bound (optional), we have:



Statistical Tools:

2. Union Bound (optional):

$$\mathbb{P}(A \text{ or } B) \leq \mathbb{P}(A) + \mathbb{P}(B)$$

Combine Hoeffding and Union Bound (optional), we have:

After the Exploration phase, with probability at least $1-\delta$, **for all**
arm $i \in [K]$, we have:

$$\left| \sum_{i=1}^N \underbrace{r_i}_{\hat{\mu}_i} / N - \mu_i \right| \leq O \left(\sqrt{\frac{\ln(K/\delta)}{N}} \right)$$

In summary, we have valid confidence intervals:

After the Exploration phase, with probability at least $1-\delta$, **for all**
arm $i \in [K]$, we have:

$$\left| \hat{\mu}_i - \mu_i \right| \leq O \left(\sqrt{\frac{\ln(K/\delta)}{N}} \right)$$

In summary, we have valid confidence intervals:

After the Exploration phase, with probability at least $1-\delta$, **for all**
arm $i \in [K]$, we have:

$$|\hat{\mu}_i - \mu_i| \leq O\left(\sqrt{\frac{\ln(K/\delta)}{N}}\right)$$

